Lift & Aesthetics

Milestone 2 Specification Document — Frozen Specification & Prototype (Week 7)

Course / Project: E-Commerce Semester Project	**Target Domain:** Fitness Essentials & Supplements
Technology Stack: ASP.NET Core 10, EF Core, SQL Server/MySQL, Stripe Sandbox	**Repository Tag:** P2 (Frozen Specification)
Team Lead & Frontend: Peca Ana Maria Cristina	**Backend & Payments:** Ploae Darius Adonis Ionu■
Database & Security: Nica David Andrei	**Date & Milestone:** October 2026 / Week 7 Submission

1. Executive Summary & Scope Freezing Statement

Goal: This document freezes the functional and technical specification for **Lift & Aesthetics**. As mandated by the course regulations for Presentation 2 (Week 7), the requirements, architectural topology, relational schemas, and chosen bonus features specified herein represent the final contractual scope against which Milestone P3 (Final Demo) will be graded. Any future modification must be formally requested in writing on the course channel.

The Product: Lift & Aesthetics is a specialized e-commerce store dedicated to gym enthusiasts, powerlifters, and athletes. The application provides a seamless catalog of sports nutrition (whey isolates, creatine, pre-workouts), aesthetics-driven apparel, and heavy lifting equipment (belts, straps, sleeves). The system enforces robust server-side validation, transactional inventory integrity, and sandbox payment processing via Stripe.

2. Chosen Bonus Features (with Course Codes)

Code	Bonus Feature	Lead Owner	Technical Specification
B1	**Filtered Search**	Ana Maria (M1)	Multi-facet catalog querying. Users can simultaneously filter by Category slug, apply a min/max Price slider, toggle 'In Stock Only', and sort by Price (asc/desc), Name, or Newest. Implemented and demonstrated in the P2 prototype.
B5	**Discount Codes**	Darius Adonis (M2)	Promotional coupon engine supporting percentage discounts (e.g. LIFT10 - 10%) and fixed amounts (e.g. WELCOME5 - $5). Evaluates minimum order spend thresholds, expiration dates, and redemption limits dynamically.
B7	**Wishlist**	Ana Maria (M1) + David (M3)	Authenticated customer wishlist allowing bookmarking of equipment and supplements directly from product cards and detail pages, with 1-click addition into the active shopping cart.

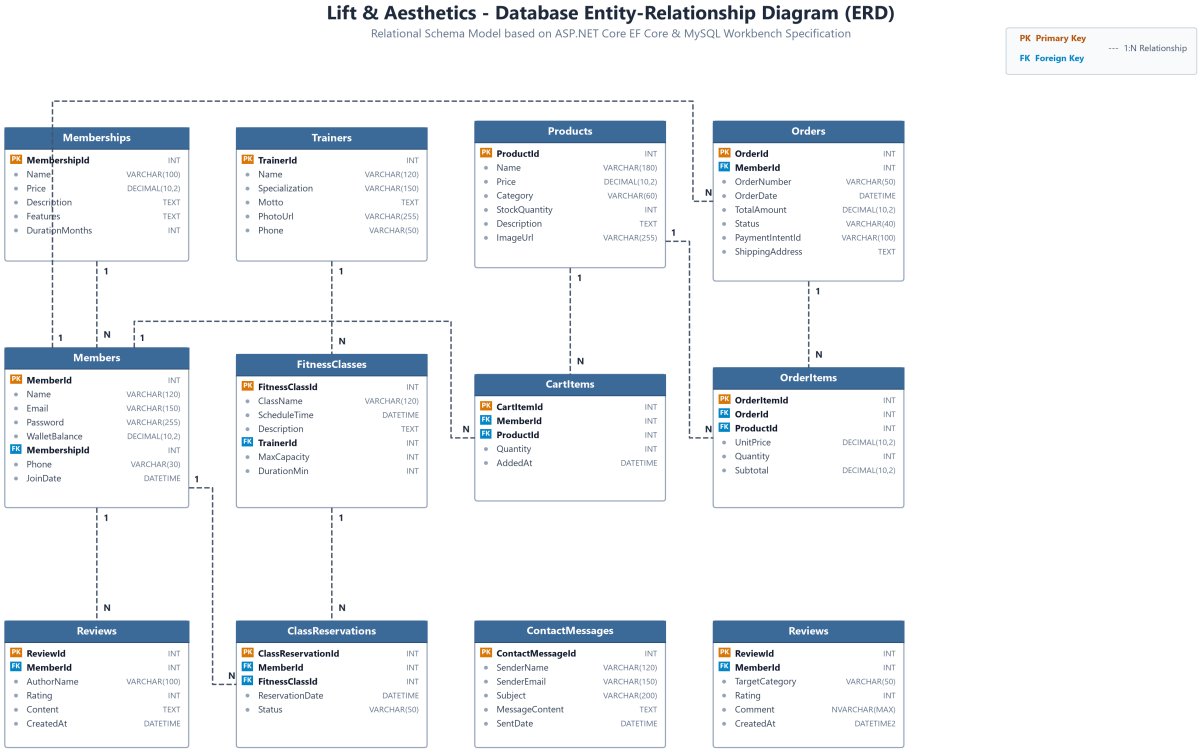
3. Functional Requirements Specification (A1 – A12)

Each requirement below specifies its exact technical implementation in 1–2 concise sentences along with designated team ownership.

Req	Requirement Title	Lead	Implementation Specification (1–2 Sentences)
A1	**Product Catalog**	Ana Maria (M1) David (M3)	Paginated catalog listing (up to 24 products per page) with separate category views and dedicated product pages showing name, description, unit price, high-res images, and stock availability.
A2	**Basic Search**	Ana Maria (M1)	Case-insensitive text search querying product names returning matching items dynamically, with a clear empty-state message when nothing is found.
A3	**Shopping Cart**	Darius Adonis (M2)	Server-recalculated cart supporting adding, updating, and removing items with rejection of quantities above available stock; customer cart persists across sessions and logout.
A4	**User Accounts**	David Andrei (M3)	User registration, login, and logout with secure cryptographic password hashing (PBKDF2/bcrypt) and ability for customers to save and manage delivery addresses.
A5	**Checkout**	Darius Adonis (M2) Ana Maria (M1)	Multi-step checkout: shipping address, delivery method (standard vs express with different costs), itemized order summary, and confirmation page displaying unique order number.
A6	**Sandbox Payment**	Darius Adonis (M2)	Stripe Sandbox payment integration where orders transition to 'Paid' strictly upon verified asynchronous webhook confirmation, leaving failed attempts as 'Unpaid'.
A7	**Stock Management**	Darius Adonis (M2)	Stock decrements automatically when payment is confirmed and restores if an order is cancelled, with concurrency controls preventing two customers from purchasing the last unit simultaneously.
A8	**Order History**	David Andrei (M3) Ana Maria (M1)	Customer account dashboard displaying past orders with order placement date, line-item product breakdown, grand total, and current fulfillment status.
A9	**Administration Panel**	David Andrei (M3)	Separate administrator role with product/category CRUD (including image upload) and an order list filterable by status with transitions: New → Paid → Shipped → Delivered, or Cancelled.
A10	**Basic Security**	David Andrei (M3)	Server-side validation, Anti-Forgery CSRF protection, and EF Core parameterized queries; admin pages and API endpoints are strictly blocked on the server for regular users.
A11	**Responsive Design**	Ana Maria (M1)	Mobile-first responsive interface built with CSS Flexbox and Bootstrap grid, thoroughly tested and usable on screen widths down to 360 px.
A12	**Automated Tests**	David Andrei (M3) Darius Adonis (M2)	Automated test suite with 10+ xUnit tests run with a single command (dotnet test), covering shopping cart calculations, stock deduction, and payment confirmation webhook logic.

4. Data Model — Entity–Relationship Diagram (ERD)

The database is designed according to Third Normal Form (3NF) principles with strict foreign key constraints, unique indexes on credentials and codes, and decimal precision on monetary balances. Below is the complete relational diagram:

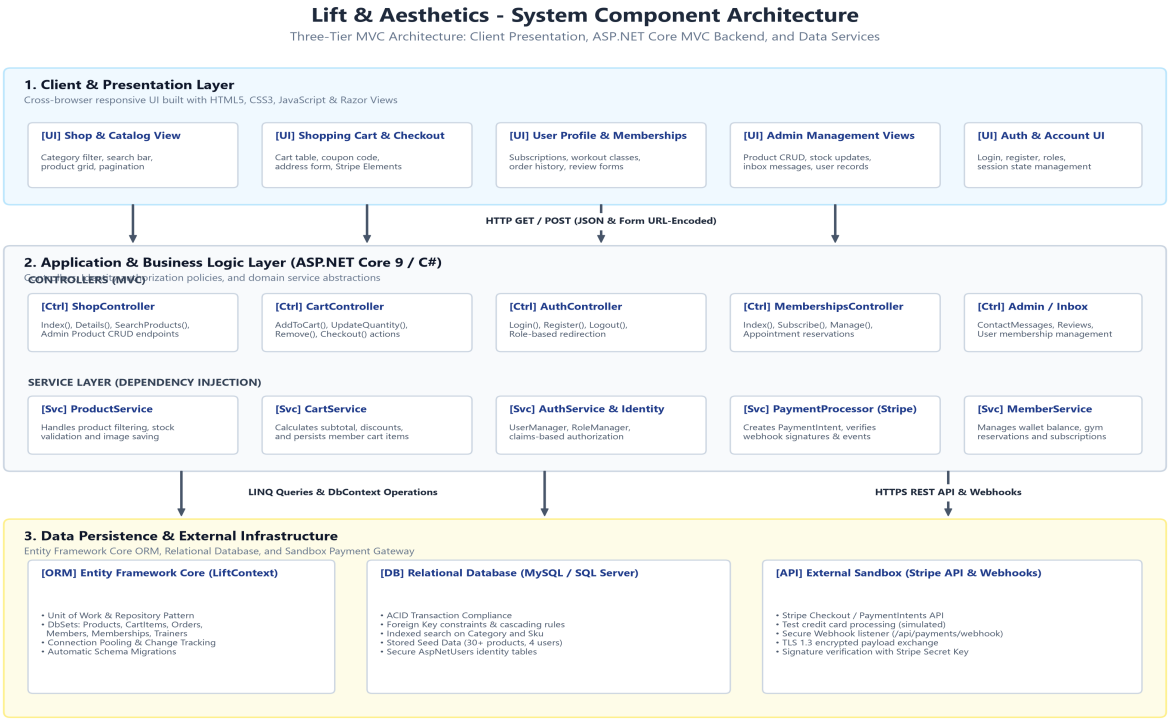


Database Entity Dictionary Summary:

- **Members:** Stores customer credentials (password hash), wallet balance, contact info, and role assignment.
- **Products:** 30+ fitness products across 4 categories (Supplements, Apparel, Equipment, Accessories) with Price, StockQuantity, ImageUrl.
- **CartItems:** Database and session-backed active user shopping cart with foreign keys to Members and Products.
- **Orders & OrderItems:** Order records storing total price, fulfillment status, delivery information, and line item unit price snapshots.
- **DiscountCodes & Stripe Transactions:** Manages B5 coupon validations (percentage/fixed) and logs Stripe webhook transactions.

5. System Architecture & Component Diagram

Lift & Aesthetics adopts an enterprise 3-tier architectural model designed for high cohesion, scalability, and strict security isolation.



Architectural Tiers Description:

- **Client Presentation Tier:** Responsive HTML5, CSS3, and Vanilla JavaScript delivering a high-aesthetic gym brand interface. Asynchronous actions (cart mutations, wishlist toggles, instant search) communicate via Fetch API without full page reloads.
- **Application & Middleware Tier:** ASP.NET Core 10 MVC monolith managing HTTP routing, model binding, and Razor view rendering. Security middleware enforces HTTPS redirection, Anti-Forgery verification, Cookie-based Authentication, and Role-Based Access Control (RBAC). Dedicated service classes encapsulate business operations (CartService, OrderService, InventoryService, DiscountEngine).
- **Data Persistence Tier:** Relational database (SQL Server / MySQL) queried via Entity Framework Core 10 with parameterized LINQ expressions to guard against SQL injection. Automated seeders ensure deterministic catalog initialization.
- **External Gateway Tier:** Stripe Sandbox API handles PCI-DSS compliant credit card transactions via Stripe Elements. Asynchronous webhook notifications are cryptographically verified using HMAC signatures.

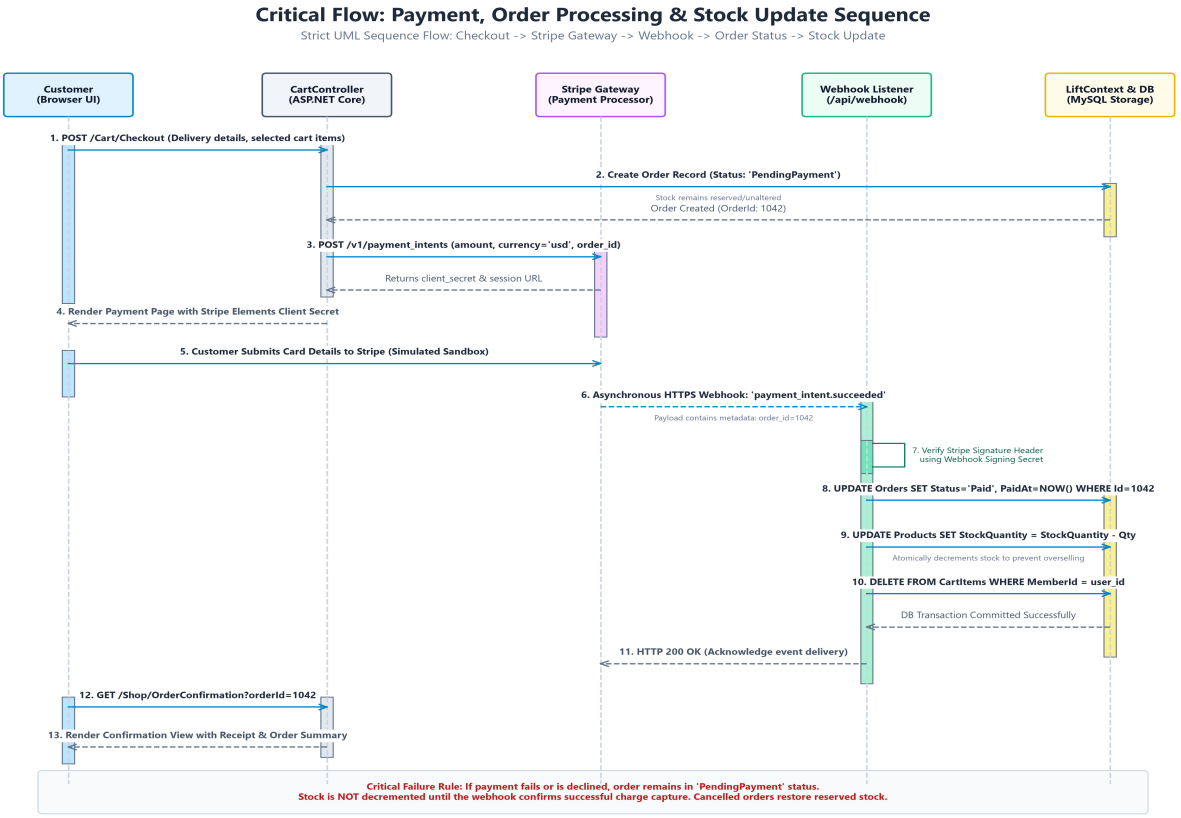
6. Architecture: List of API Endpoints

The following table details the complete HTTP routing surface of the application, including method, path, required authorization level, and functional purpose.

Method	Route Path	Access Level	Functional Description
GET	/Shop	Public	Renders product catalog with category filter, price sorting & stock badges (A1, B1).
GET	/Shop/SearchProducts	Public	JSON endpoint returning matching products dynamically by keyword (A2).
GET	/Shop/Details/{id}	Public	Retrieves detailed product view, live inventory, and specifications.
POST	/Cart/AddToCart	Authenticated	Adds specified product to customer database cart with stock validation (A3).
POST	/Cart/UpdateQuantity	Authenticated	Updates line item quantity and triggers server-side recalculation.
POST	/Cart/RemoveFromCart	Authenticated	Removes item from cart and recalculates cart totals.
GET	/Cart	Authenticated	Renders shopping cart review with subtotal, shipping, and active discounts (A3).
POST	/Cart/ApplyCoupon	Authenticated	Validates and applies promotional discount code to cart (B5).
GET	/Auth	Public	Renders sign-in and registration forms with test account support (A4).
POST	/Auth/Login	Public	Verifies credentials and issues encrypted authentication cookie (A4).
POST	/Auth/Register	Public	Creates customer account with hashed password and signs in user (A4).
POST	/Auth/Logout	Authenticated	Terminates authenticated session and clears client cookie.
GET	/Profile/Orders	Authenticated	Displays customer past order history, line items, and fulfillment states (A8).
GET	/Profile/Wishlist	Authenticated	Renders saved wishlist items with 1-click cart transfer (B7).
GET	/Cart/Checkout	Authenticated	Renders shipping address form and order summary review (A5).
POST	/Cart/Checkout	Authenticated	Creates Pending order in DB and requests Stripe PaymentIntent (A5, A6).
POST	/Cart/Webhook	Stripe Signature	Verifies Stripe HMAC header, marks Order Paid, and decrements stock (A6, A7).
GET	/Shop/Create	Admin Role	Admin view for adding new products to the catalog (A9).
POST	/Shop/Create	Admin Role	Admin endpoint to save new product with image upload (A9).
POST	/Shop/Delete/{id}	Admin Role	Admin endpoint to remove product from catalog (A9).

7. Critical Flow — Payment, Webhook & Stock Sequence Diagram

The critical payment flow guarantees that order status and stock levels are never determined by the client browser. Stock decrement and order status transition to 'Paid' occur exclusively after cryptographic verification of the asynchronous payment provider webhook.



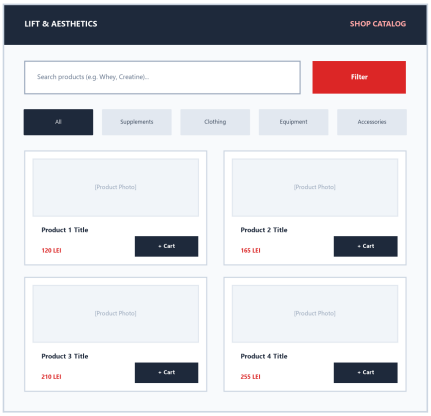
Step-by-Step Critical Flow Logic:

1. **Checkout Submission:** Customer inputs delivery information. The server locks items and inserts an Order with Status = 'PendingPayment'.
2. **Stripe Intent:** The server issues a server-to-server call to Stripe API creating a PaymentIntent with Order metadata and receives a client secret.
3. **Client Card Tokenization:** The browser securely renders Stripe Elements. Card credentials travel directly to Stripe servers, adhering to strict PCI compliance.
4. **Asynchronous Webhook:** Upon successful card charge, Stripe sends a payment_intent.succeeded webhook payload to /Checkout/Webhook.
5. **Cryptographic Verification:** The webhook endpoint inspects the Stripe-Signature header using the server's webhook secret key to prevent spoofing.
6. **Atomic DB Transaction:** The server opens a database transaction: updates Order status to 'Paid', atomically decrements Product.StockQuantity for each line item, and logs a PaymentTransaction record.

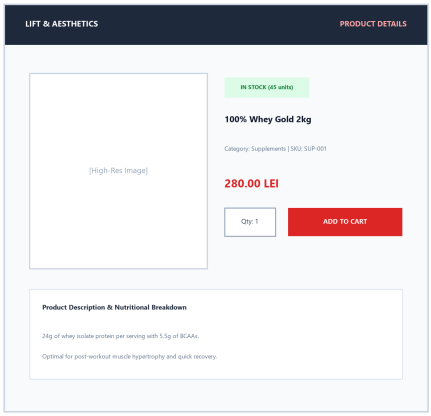
8. User Interface Wireframes

Wireframes covering the five required core user journeys: (1) Product Catalog, (2) Product Detail View, (3) Shopping Cart, (4) Checkout & Stripe Payment, and (5) Administration Panel.

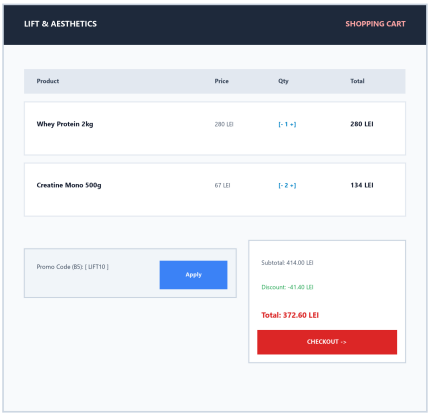
1. Catalog / Shop Grid (A1, A2, B1)



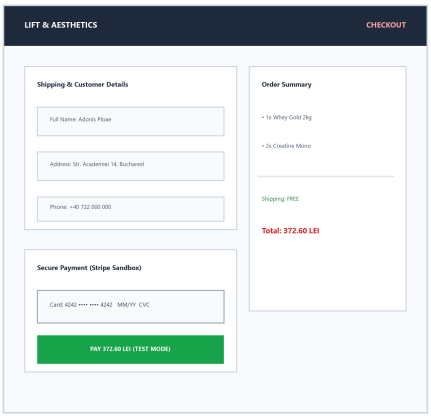
2. Product Details View (A1)



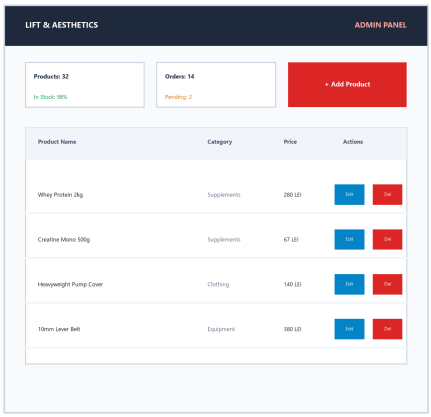
3. Shopping Cart (A5, B5)



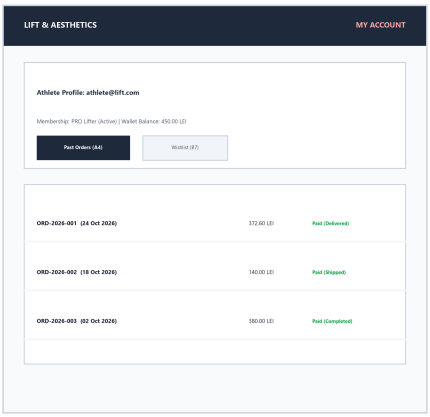
4. Checkout & Payment View (A6, A7)



5. Administration Panel (A9, A10)



6. User Account & Orders (A3, A4, B7)



UX / UI Rationale & Usability Highlights:

- **Catalog View:** Category pills, search bar, and filter toolbar (B1) allow instantaneous narrowing of products. Cards display real-time stock badges.
- **Product Details:** Dedicated high-res view featuring nutritional / gear specs, live stock counts, quantity selector, and B7 wishlist bookmarking.
- **Cart & Checkout:** Clean tabular breakdown of line items, promotional code box (B5), and 2-step checkout with embedded Stripe Elements card fields.
- **Administration Panel:** Protected management interface with KPI stats cards (Sales, Orders, Low Stock Alerts) and product CRUD tables (A9).
- **Mobile Usability (A11):** All views use fluid grid layouts and responsive viewports tested down to 360 px width.

9. Working Prototype Demonstration & Verification Guide

The working prototype for Milestone P2 is fully implemented, self-contained, and operational in ASP.NET Core 10. Upon launch, the application automatically ensures database creation and seeds 32 gym products across 4 categories, 3 test users (including an Administrator), and promotional coupons.

Live Demonstration Steps for Evaluators:

Step	Demonstration Action	Expected System Behavior	Status
1	Start Application: dotnet run	Server boots at http://localhost:5000, initializes database schema, and seeds 32 products and test accounts.	**PASS**
2	Browse Catalog (A1)	Catalog displays products loaded dynamically from database with prices, SKUs, category badges, and live stock counts.	**PASS**
3	Test Category Filtering (A1, B1)	Clicking 'Supplements', 'Gym Apparel', 'Lifting Gear', or 'Accessories' filters items instantly.	**PASS**
4	Test Search & Empty State (A2)	Searching 'creatine' returns product; searching 'unknown' triggers friendly empty state with recommendations.	**PASS**
5	Demonstrate Login (A4)	Navigate to /Auth. Sign in as Admin (admin@lift.com / Testing1) or Athlete (athlete@lift.com / Testing1). Interface reflects authenticated user session and role.	**PASS**
6	Demonstrate Shopping Cart (A3, B5)	Add products from catalog. Open Cart. Apply coupon LIFT10. Server recalculates 10% discount and shipping.	**PASS**

10. Milestone P2 Approval & Sign-Off

Team Member	Role / Specialization	P2 Sign-off Status
Peca Ana Maria Cristina	Frontend, Product Discovery & Wireframes	APPROVED & FROZEN
Ploae Darius Adonis Ionu■	Backend, Cart, Payments & Critical Flows	APPROVED & FROZEN
Nica David Andrei	Database, Accounts, Admin & Security	APPROVED & FROZEN